

DBMS LECTURE 3: TOPPER'S NOTES

Topic: 2-Tier & 3-Tier Architecture | Quick Revision & Exam Prep

1. INTRODUCTION

Tier simply means a **Layer**.

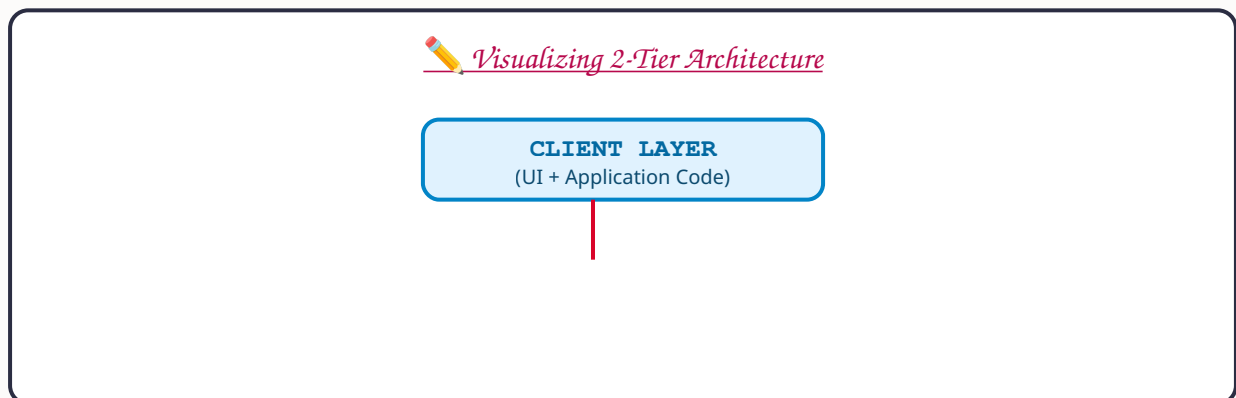
- Database applications are structured in multiple layers so that users, application logic, and databases communicate systematically.
- Core Difference: Whether application/business logic is placed directly on the client machine or in a separate middle layer.

2. 2-TIER ARCHITECTURE

Definition

An architecture where the client **directly communicates** with the database server.

- Also called: **Client-Server Architecture**
- Formula: **2-Tier = Client + Database Server**



Components of 2-Tier Architecture

Tier 1 — Client: Contains UI + Application Code + Connectivity drivers (JDBC/ODBC). Sends requests directly to DB server.

Tier 2 — Database Server: Receives requests, processes queries, updates/retrieves data, and returns results back.

Real-Life Examples

Bank Branch Counter: Bank employee enters account # → Client App directly queries DB server → Verifies balance & processes withdrawal.

Railway Reservation Counter: Counter staff enters passenger details → Client app queries DB server directly for seat availability.

3. PROS & CONS OF 2-TIER ARCHITECTURE

Advantages

- *Simple Architecture*: Only 2 layers to set up.
- *Fast Performance*: Direct connection = minimal network overhead for small systems.
- *Easy Development*: Straightforward to code for small internal applications.

Disadvantages

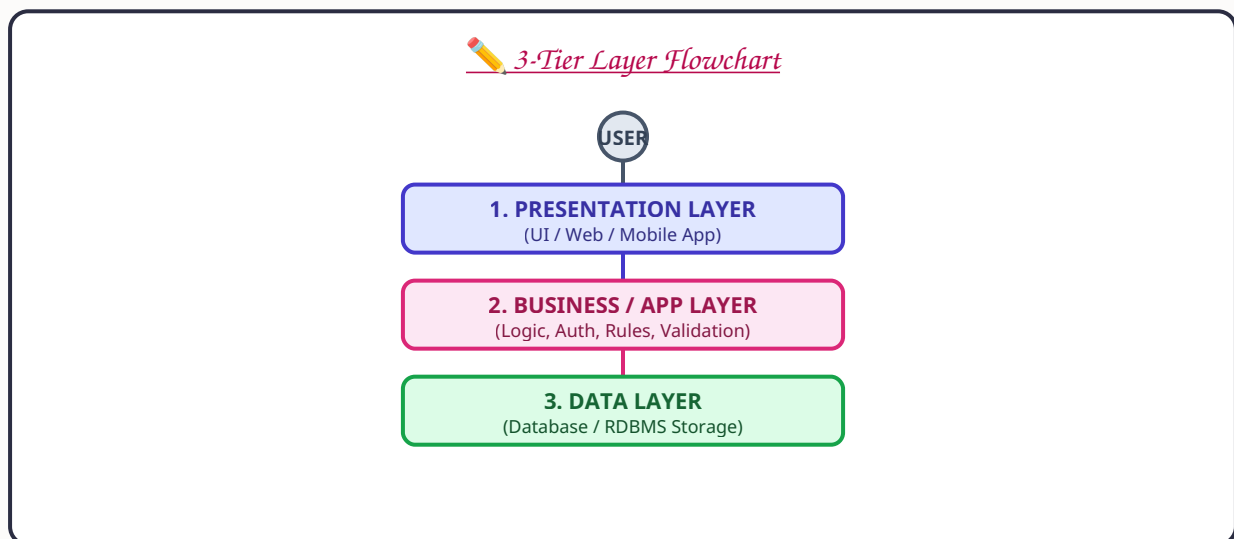
- **Scalability Problem** ★ Server overloads when thousands of clients connect directly.
- **Security Risk** ★ Direct DB exposure increases vulnerability to attacks.
- **High Maintenance** ★ Updating application logic requires updating every single client machine.

4. 3-TIER ARCHITECTURE

Definition

Divides the application into **3 separate layers**: Presentation Layer, Application/Business Layer, and Data Layer.

Formula: **3-Tier = Presentation + Business + Data**



The Three Layers in Detail

- Tier 1 — *Presentation Layer*: User interface (web page, mobile app, desktop GUI). Collects user inputs (forms, clicks) and displays results.
- Tier 2 — *Application / Business Layer*: The middle layer executing business rules, calculations, input validation, and access control. Client NEVER directly touches DB!
- Tier 3 — *Data Layer*: Manages actual data storage, DBMS execution, tables, and records.



WHY DO WE NEED THE BUSINESS LAYER? ★★ ★

In web apps with 10,000+ simultaneous users, direct DB connections would crash the system. The Business Layer acts as a buffer using Connection Pooling, Caching, and Load Balancing, while preventing direct access to sensitive data.

5. KEY BENEFITS OF 3-TIER ARCHITECTURE

- **Scalability** ★★ ★ App servers can be scaled independently using a Load Balancer.
- **Security** ★★ ★ Database is isolated behind the app layer; rules & authorization are checked first.
- **Maintainability:** Centralized business logic — change rules on the app server without re-deploying client apps.
- **Reusability:** One business layer can serve Web, Mobile, Desktop apps, and 3rd-party APIs simultaneously!

6. COMPARISON: 2-TIER vs 3-TIER ARCHITECTURE

Feature	2-Tier Architecture	3-Tier Architecture
Number of Layers	2 Layers	3 Layers
Main Structure	Client + Database Server	Presentation + Business + Data
Direct DB Access?	YES	NO (via Business Layer)
Scalability	Lower (DB bottleneck)	Higher (Independent app scaling)
Security	Weaker exposure	Better isolation & access control
Maintenance	Hard across many clients	Easier (Centralized app logic)
Best Suited For	Small / Controlled systems	Large distributed web systems



EXAM CHEAT SHEET & MEMORY TRICKS

- ★ 1-Line Difference: 2-Tier is Client ↔ DB | 3-Tier is Client ↔ Business Layer ↔ DB.
- ★ Exam Tip: If asked why 3-tier is more scalable, write: "Application servers can be scaled horizontally using load balancers independently of the database layer."
- ★ Common Mistake: Don't call 2-tier obsolete! It's still used in small/internal tools like local bank branch counters.